Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_07_)

Implementation of Arithmetic Logic Unit (ALU)

Student Name: ___Areeb Fatima____ Roll No: _____24I-6543_____ Section: _B__

Date performed: ___7th October_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task, we done the selection operation by using mux and we perform the three operation by using 2 selection lines and done this for add, sub, and AND and OR also.So, basically we import the modules from pervious labs and verify the output for every possible combination.

Code:

Gate-Level Modeling:

```
//And Gate

module AndGate(a,b,c);

input a;

input b;

output c;

and a1(c,a,b);

endmodule

//Or Gate:

module OrGate(a,b,c);

input a;

input b;

output c;

or a2(c,a,b);

endmodule

//HALF ADDER (Gate level):

module half_adder(A,B,sum,carry);

input A,B;
```

```
output sum,carry;

xor(sum,A,B);

and(carry,A,B);

endmodule

//FULL ADDER (Gate Level):

module full_Adder(a,b,CIN,SUM,CARRY);

input a,b,CIN;

output SUM,CARRY;

wire t1,t2,t3;

half_adder ha1(a,b,t1,t2);

half_adder ha2(t1,CIN,SUM,t3);

or o1(CARRY,t2,t3);

endmodule

//MUX_2X1:

module MUX_2x1(b,b_not,s,Y);

input b, b_not, s;

output Y;

wire y0, y1;

wire ns;

not n0(ns, s);

and g1(y0, b, ns); //0

and g2(y1, b_not, s); //1

or g5(Y, y0, y1);
```

```
endmodule

//MUX_4X1:

module MUX_4x1(And,Or,adder,select,Y);

input And, Or, adder;

input [1:0] select;

output Y;

wire y0, y1, y2;

wire ns0, ns1;

not n0(ns0, select[0]);

not n1(ns1, select[1]);

and g1(y0, And, ns1, ns0); //00

and g2(y1, Or, ns1, select[0]); //01

and g3(y2, adder, select[1], ns0); //10

or g5(Y, y0, y1, y2);

endmodule

module OneBit_Adder(A, B, b_inv, carry_in, select, result, carry_out);

input A, B, b_inv, carry_in;

input [1:0] select;

output result, carry_out;

wire B_not, B_out, and_result, or_result, sum_result, actual_carry;

not n1(B_not, B);

MUX_2x1 m1(B, B_out, b_inv, B_out);

AndGate a3(A, B_out, and_result);
```

```
OrGate o1(A, B_out, or_result);

full_Adder f1(A, B_out, carry_in, sum_result, actual_carry);

MUX_4x1 m2(and_result, or_result, sum_result, select, result);

// Carry-out only for ADD/SUB

assign carry_out =(select == 2'b10) ? actual_carry : 1'b0;

endmodule

//Common Testbench:

module tb_OneBit_Adder();

reg x, y, y_inv, c_in;

reg [1:0] s;

wire result, c_out;

OneBit_Adder uut(x, y, y_inv, c_in, s, result, c_out);

initial

begin

// 1. Test AND (s = 2'b00)

s = 2'b00; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

// 2. Test OR (s=2'b01)

s = 2'b01; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;
```

```
x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

// 3. Test ADD (s=2'b10, y_inv=0)

s = 2'b10; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

// 4. Test SUB (s=2'b10, y_inv=1, c_in=1) -> A-B= A+~B+1

s = 2'b10; y_inv = 1; c_in = 1;

x = 0; y = 0; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

x = 0; y = 1;

end

endmodule
```

Dataflow Modeling:

```
module OneBit_ALU_dataflow (

   input a,

   input b,

   input carry_in,

   input b_inv,
```

```
    input [1:0] select,
    output result,
    output carry_out
);
    wire b_mux;
    wire and_res, or_res, sum_res;
    // B Inversion MUX
    assign b_mux = b_inv ? ~b : b;
    // Logic Functions
    assign and_res = a & b_mux;
    assign or_res  = a | b_mux;
    // Full Adder Dataflow Logic
    assign {carry_out, sum_res} = a + b_mux + carry_in;
    // 4-to-1 Operation Multiplexer
    assign result = (select == 2'b00) ? and_res :
                    (select == 2'b01) ? or_res  :
                    (select == 2'b10) ? sum_res : 1'b0;
endmodule
//Common Testbench:
module tb_OneBit_ALU();
reg x, y, y_inv, c_in;
reg [1:0] s;
wire result, c_out;
```

```
OneBit_ALU_dataflow uut(x, y, c_in, y_inv, s, result, c_out);

initial

begin

s = 2'b00; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

s = 2'b01; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

s = 2'b10; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

s = 2'b10; y_inv = 1; c_in = 1;

x = 0; y = 0; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

x = 0; y = 1;
```

```
end

endmodule
```

Behavioral-Level Modeling:

```
module alu_1bit_behavioral (
    input a,
    input b,
    input carry_in,
    input b_inv,
    input [1:0] select,
    output reg result,
    output reg carry_out
);
    reg b_mux;
    reg sum_res;
    always @(*) begin
        if (b_inv)
            b_mux = ~b;
        else
            b_mux = b;
        {carry_out, sum_res} = a + b_mux + carry_in;
        case (select)
            2'b00: result = a & b_mux; // AND
            2'b01: result = a | b_mux; // OR
```

```
        2'b10: result = sum_res;   // ADD / SUB

        default: result = 1'b0;

      endcase

   end

endmodule



//Common Testbench:

module tb_OneBit_ALU();

reg x, y, y_inv, c_in;

reg [1:0] s;

wire result, c_out;

alu_1bit_behavioral uut(x, y, c_in, y_inv, s, result, c_out);

initial

begin

s = 2'b00; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

s = 2'b01; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;
```

```
x = 1; y = 1; #50;

s = 2'b10; y_inv = 0; c_in = 0;

x = 0; y = 0; #50;

x = 0; y = 1; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

s = 2'b10; y_inv = 1; c_in = 1;

x = 0; y = 0; #50;

x = 1; y = 0; #50;

x = 1; y = 1; #50;

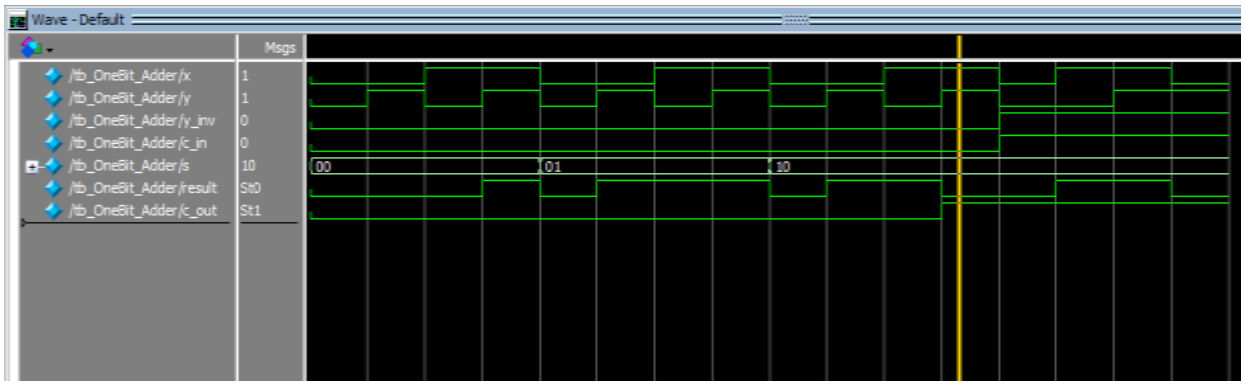
x = 0; y = 1;

end

endmodule
```

Output:

Gate-Level Modeling:

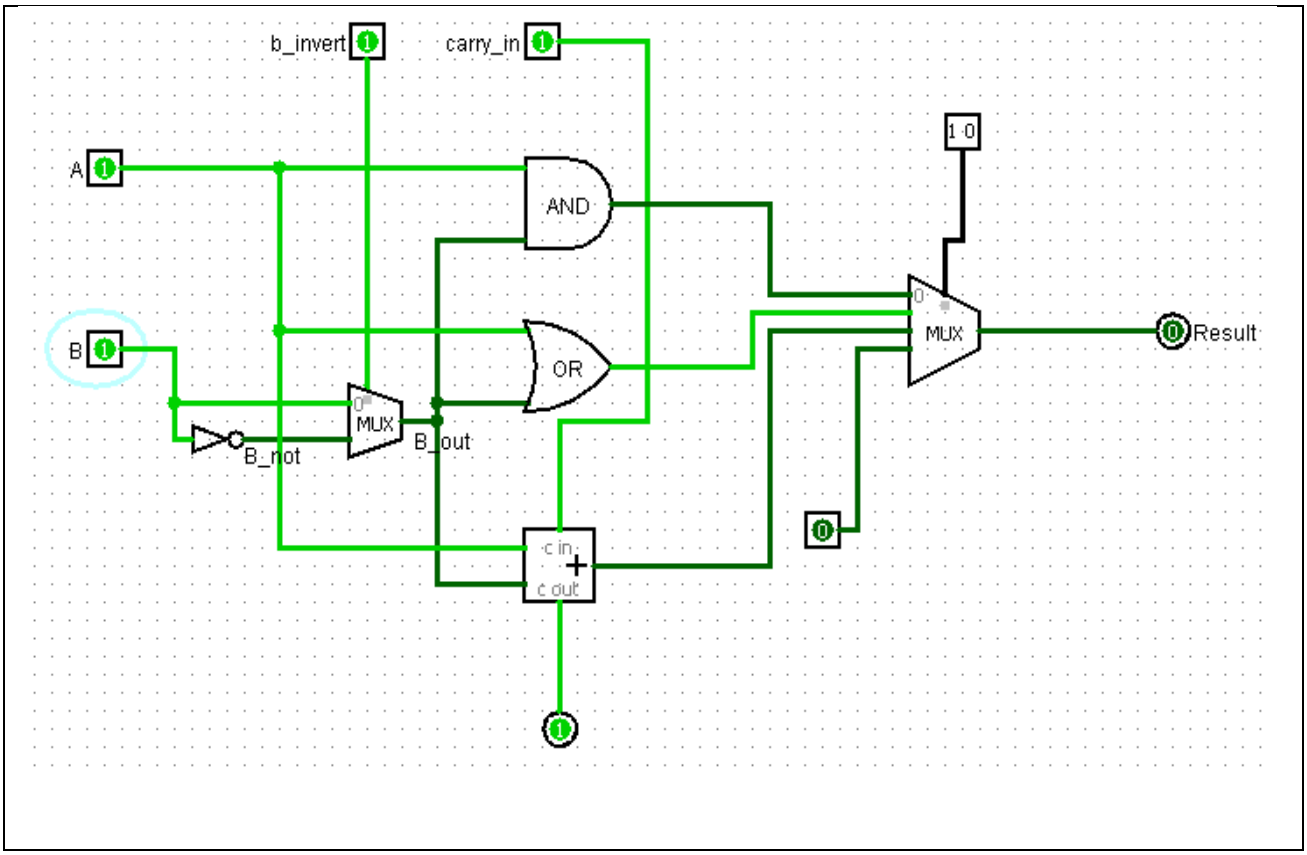


Dataflow Modeling:



Behavioral-Level Modeling:





Task 2:

Description:
In this task, we done the selection operation by using mux and we perform the four operation by using 2 selection lines and done this for add, sub, and set less than also.
Code:

//And Gate
 	
module AndGate(a,b,c);
 	
input a;
 	
input b;
 	
output c;
 	
and a1(c,a,b);

```
endmodule

//Or Gate:

module OrGate(a,b,c);

input a;

input b;

output c;

or a2(c,a,b);

endmodule

//HALF ADDER (Gate level):

module half_adder(A,B,sum,carry);

input A,B;

output sum,carry;

xor(sum,A,B);

and(carry,A,B);

endmodule

//FULL ADDER (Gate Level):

module full_Adder(a,b,CIN,SUM,CARRY);

input a,b,CIN;

output SUM,CARRY;

wire t1,t2,t3;

half_adder ha1(a,b,t1,t2);

half_adder ha2(t1,CIN,SUM,t3);

or o1(CARRY,t2,t3);
```

```
endmodule

//MUX_2X1:

module MUX_2x1(b,b_not,s,Y);

input b, b_not, s;

output Y;

wire y0, y1;

wire ns;

not n0(ns, s);

and g1(y0, b, ns); //0

and g2(y1, b_not, s); //1

or g5(Y, y0, y1);

endmodule

//MUX_4X1:

module MUX_4x1(And,Or,adder,less,select,Y);

input And, Or, adder, less;

input [1:0] select;

output Y;

wire y0, y1, y2,y3;

wire ns0, ns1;

not n0(ns0, select[0]);

not n1(ns1, select[1]);

and g1(y0, And, ns1, ns0); //00

and g2(y1, Or, ns1, select[0]); //01
```

```
and g3(y2, adder, select[1], ns0); //10

and g4(y3, less, select[1], select[0]); //11

or g5(Y, y0, y1, y2, y3);

endmodule

module OneBit_ALU(A, B, b_inv, carry_in, select, less, result, carry_out, sum_result);

input A, B, b_inv, carry_in, less;

input [1:0] select;

output result, carry_out, sum_result;

wire B_not, B_out, and_result, or_result, sum_result;

not n1(B_not, B);

MUX_2x1 m1(B, B_not, b_inv, B_out);

AndGate a3(A, B_out, and_result);

OrGate o1(A, B_out, or_result);

full_Adder f1(A, B_out, carry_in, sum_result, carry_out);

MUX_4x1 m2(and_result, or_result, sum_result, less, select, result);

endmodule

module alu_4bit(A, B, b_inv, select, Result, CF, SF, ZF, OF);

input [3:0] A, B;

input b_inv;

input [1:0] select;

output [3:0] Result;

output CF, SF, ZF, OF;

wire [3:0] c_out;
```

```
wire [3:0] sum_out;

wire set_signal;

wire initial_cin;

// Subtraction ya SLT (select = 11) ke liye CarryIn = 1 hona chahiye

assign initial_cin = (select == 2'b11) ? 1'b1 : ((select == 2'b10) ? b_inv : 1'b0);

// MSB Adder ka Sum result acting as Set signal for SLT

assign set_signal = sum_out[3];

// ALU 0 (LSB) - receives 'set_signal' at 'less'

OneBit_ALU alu0(A[0], B[0], b_inv, initial_cin, select, set_signal, Result[0], c_out[0], sum_out[0]);

// ALU 1, 2, 3 - receive 1'b0 at 'less'

OneBit_ALU alu1(A[1], B[1], b_inv, c_out[0], select, 1'b0, Result[1], c_out[1], sum_out[1]);

OneBit_ALU alu2(A[2], B[2], b_inv, c_out[1], select, 1'b0, Result[2], c_out[2], sum_out[2]);

OneBit_ALU alu3(A[3], B[3], b_inv, c_out[2], select, 1'b0, Result[3], c_out[3], sum_out[3]);

// Status Flags Calculation

assign CF = c_out[3];  // Carry Flag

assign SF = Result[3]; // Sign Flag

assign ZF = ~(Result[0] | Result[1] | Result[2] | Result[3]); // Zero Flag (NOR Gate)

assign OF = c_out[2] ^ c_out[3]; // Overflow Flag

endmodule

//Common Testbench:

module tb_alu_4bit();

reg [3:0] x, y;

reg y_inv;
```

```
reg [1:0] s;

wire [3:0] result;

wire cf, sf, zf, of;

alu_4bit uut(

   .A(x), .B(y), .b_inv(y_inv), .select(s),.Result(result),

.CF(cf), .SF(sf), .ZF(zf), .OF(of)

);

initial

begin

   // 1. Test ADD: A = 5 (0101), B = 3 (0011) -> Result = 8 (1000)

   x = 4'b0101; y = 4'b0011; y_inv = 0; s = 2'b10; #50;

   // 2. Test SUB: A = 7 (0111), B = 2 (0010) -> Result = 5 (0101)

   x = 4'b0111; y = 4'b0010; y_inv = 1; s = 2'b10; #50;

   // 3. Test AND: A = 12 (1100), B = 10 (1010) -> Result = 8 (1000)

   x = 4'b1100; y = 4'b1010; y_inv = 0; s = 2'b00; #50;

   // 4. Test OR: A = 12 (1100), B = 10 (1010) -> Result = 14 (1110)

   x = 4'b1100; y = 4'b1010; y_inv = 0; s = 2'b01; #50;

   // 5. Test SLT (True): A = 2 (0010), B = 5 (0101) -> Result = 0001

   x = 4'b0010; y = 4'b0101; y_inv = 1; s = 2'b11; #50;

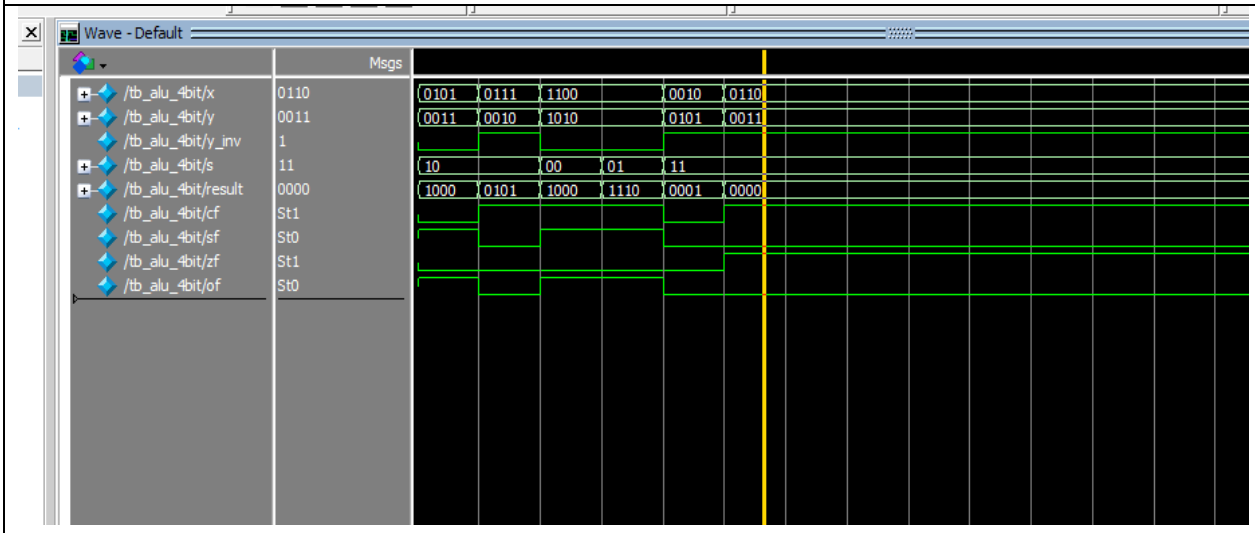
   // 6. Test SLT (False): A = 6 (0110), B = 3 (0011) -> Result = 0000

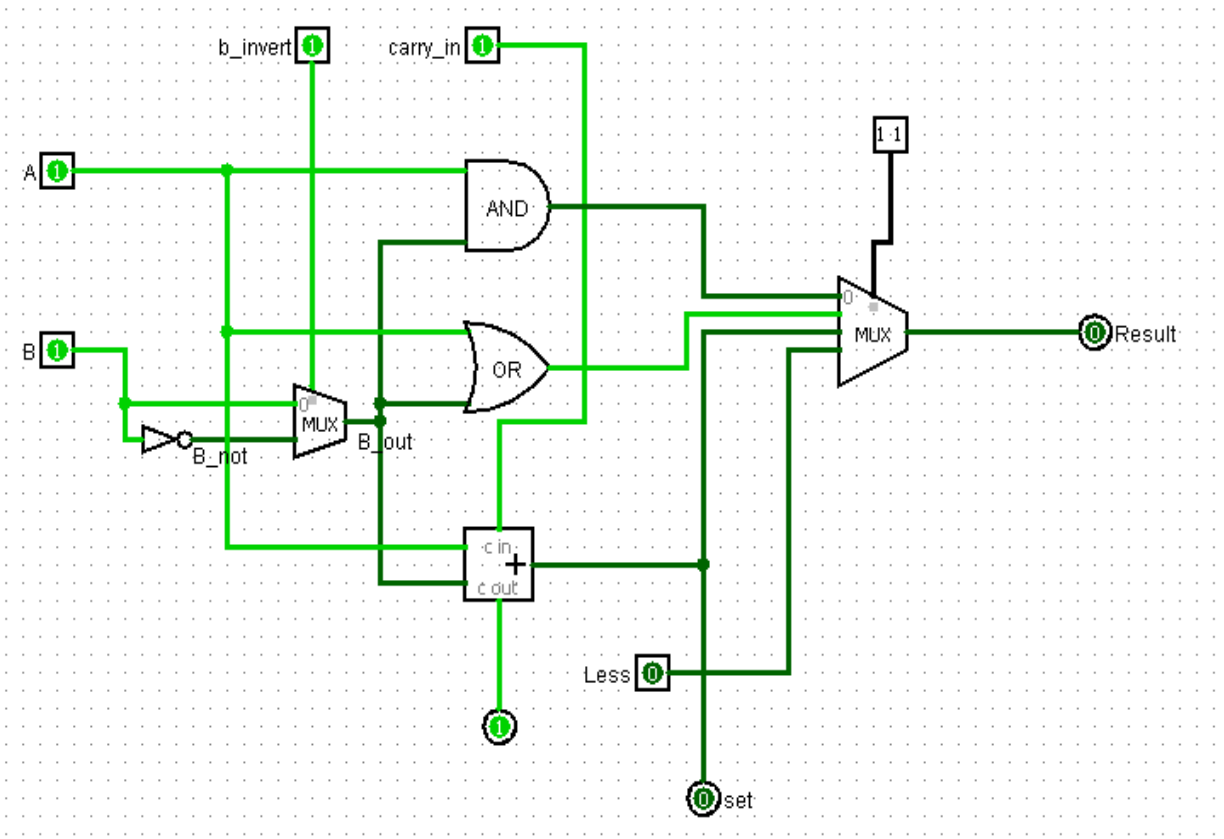
   x = 4'b0110; y = 4'b0011; y_inv = 1; s = 2'b11; #50;

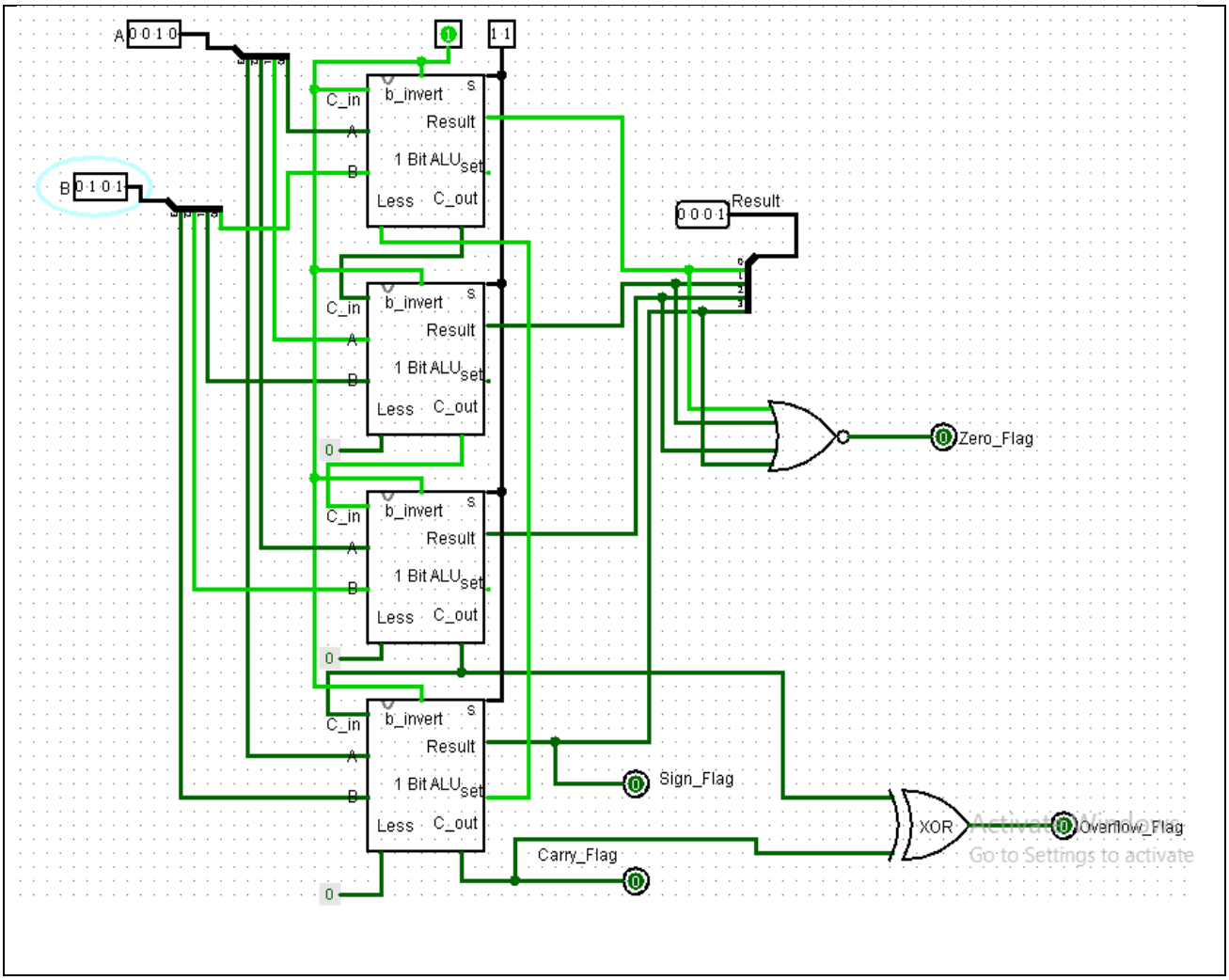
end

endmodule
```

Output:


Wave - Default
Msgs
/tb_alu_4bit/x
/tb_alu_4bit/y
/tb_alu_4bit/y_inv
/tb_alu_4bit/s
/tb_alu_4bit/result
/tb_alu_4bit/cf
/tb_alu_4bit/sf
/tb_alu_4bit/zf
/tb_alu_4bit/of
0110
0011
1
11
0000
St1
St0
St1
St0
0101 0111 1100 0010 0110
0011 0010 1010 0101 0011
10 00 01 11
1000 0101 1000 1110 0001 0000


b_invert
carry_in
A
B
AND
OR
MUX
B_not
B_out
1
MUX
Result
c in
c out
Less
set



Task 3:

Description:
In this task, we were asked to make a parameterized ALU so that it can be implemented in any of the ways and we simply import the modules from pervious labs and done this task.
Code:
module parameterized_alu #(parameter N = 4)(input wire [N-1:0] A, input wire [N-1:0] B, input wire [2:0] ALU_Sel,

```
    output reg  [N-1:0] Result

);

    localparam ADD = 3'b000;  // Add

    localparam SUB = 3'b001;  // Subtract

    localparam AND = 3'b010;  // Bitwise AND

    localparam OR  = 3'b011;  // Bitwise OR

    localparam XOR = 3'b100;  // Bitwise XOR

    localparam NOT = 3'b101;  // Bitwise NOT (Operand A)

    always @(*)

    begin

        case (ALU_Sel)

            ADD: Result = A + B;

            SUB: Result = A - B;

            AND: Result = A & B;

            OR:  Result = A | B;

            XOR: Result = A ^ B;

            NOT: Result = ~A;

            default: Result = {N{1'b0}};

        endcase

    end

endmodule



module tb_parameterized_alu();
```

```
parameter N = 8;

reg  [N-1:0] A;

reg  [N-1:0] B;

reg  [2:0]   ALU_Sel;

wire [N-1:0] Result;

parameterized_alu #(.N(N)) uut (.A(A),.B(B),.ALU_Sel(ALU_Sel),.Result(Result));

initial

begin

   A = 8'd15;   // 00001111

   B = 8'd10;   // 00001010

   // a. Test Addition (ALU_Sel = 000)

   ALU_Sel = 3'b000; #10;

   // b. Test Subtraction (ALU_Sel = 001)

   ALU_Sel = 3'b001; #10;

   // c. Test Logical AND (ALU_Sel = 010)

   ALU_Sel = 3'b010; #10;

   // c. Test Logical OR (ALU_Sel = 011)

   ALU_Sel = 3'b011; #10;

   // c. Test Logical XOR (ALU_Sel = 100)

   ALU_Sel = 3'b100; #10;

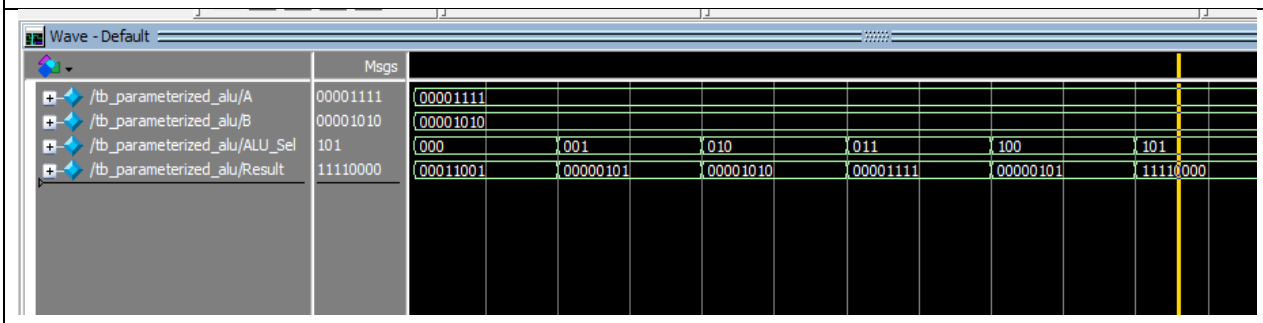
   // c. Test Logical NOT (ALU_Sel = 101)

   ALU_Sel = 3'b101; #10;

end
```

```
endmodule
```

Output:



Task 4:

Description:

In this task, we were asked to make 32-bit ALU so we done this by making the 32-bit ALU module and then doing it for all the operations and we done this by using if else in behavorial level for 32 bits.

Code:

```
module ALU_32Bit(

   input  [31:0] x,

   input  [31:0] y,

   input  [3:0] alu_op,

   output reg [31:0] result,

   output zero_flag

);

   always @(*) begin

      if (alu_op == 4'b0000)

         result = x + y; // ADD

      else if (alu_op == 4'b0001)

         result = x - y; // SUB
```

```
    else if (alu_op == 4'b0010)
       result = x & y; // AND
    else if (alu_op == 4'b0011)
       result = x | y; // OR
    else if (alu_op == 4'b0100)
       result = x ^ y; // XOR
    else if (alu_op == 4'b0101) begin // SLT(Signed)
       if (x[31] != y[31])
          result = x[31] ? 32'd1 : 32'd0;
       else
          result = (x - y) >> 31;
    end
    else if (alu_op == 4'b0110) begin // SLTU
       result = (x < y) ? 32'd1 : 32'd0;
    end
    else if (alu_op == 4'b0111)
       result = x << y[4:0]; // SLL
    else if (alu_op == 4'b1000)
       result = x >> y[4:0]; // SRL
    else if (alu_op == 4'b1001) begin // SRA
       if (x[31] == 1'b0)
          result = x >> y[4:0];
       else
```

```
            result = (x >> y[4:0]) | ~(32'hFFFFFFFF >> y[4:0]);

        end

        else

            result = 32'b0; // Default Safety Case

    end

    // Zero Flag Logic

    assign zero_flag = (result == 32'b0);

endmodule

module tb_ALU_32Bit();

    reg  [31:0] x;

    reg  [31:0] y;

    reg  [3:0]  operation;

    wire [31:0] answer;

    wire zero_flag;

    ALU_32Bit dut(.x(x),.y(y),.alu_op(operation),.result(answer),.zero_flag(zero_flag));

    initial begin

            // 1. ADD: 10 + 5

        x = 32'd10; y = 32'd5; operation = 4'b0000; #10;

        // 2. SUB: 10 - 10 (Zero Flag Test)

        x = 32'd10; y = 32'd10; operation = 4'b0001; #10;

        // 3. AND

        x = 32'h0000000A; y = 32'h0000000C; operation = 4'b0010; #10;

        // 4. OR
```

```
    x = 32'h0000000A; y = 32'h0000000C; operation = 4'b0011; #10;

    // 5. XOR

    x = 32'h0000000A; y = 32'h0000000C; operation = 4'b0100; #10;

    // 6. SLT (Signed Comparison): -5 < 3 -> True (1)

    x = -32'sd5; y = 32'd3; operation = 4'b0101; #10;

    // 7. SLTU (Unsigned Comparison): 5 < 10 -> True (1)

    x = 32'd5; y = 32'd10; operation = 4'b0110; #10;

    // 8. SLL: 3 << 2

    x = 32'd3; y = 32'd2; operation = 4'b0111; #10;

    // 9. SRL: 16 >> 2

    x = 32'd16; y = 32'd2; operation = 4'b1000; #10;

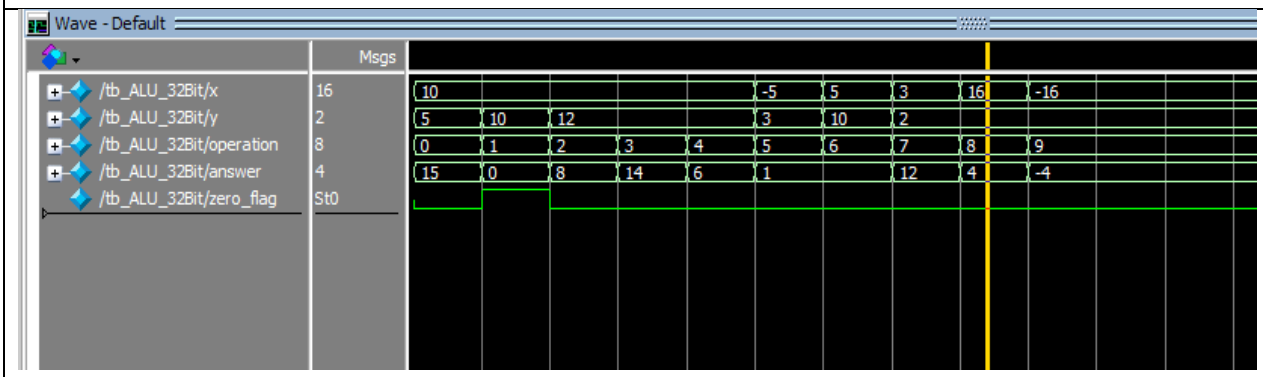
    // 10. SRA: -16 >>> 2

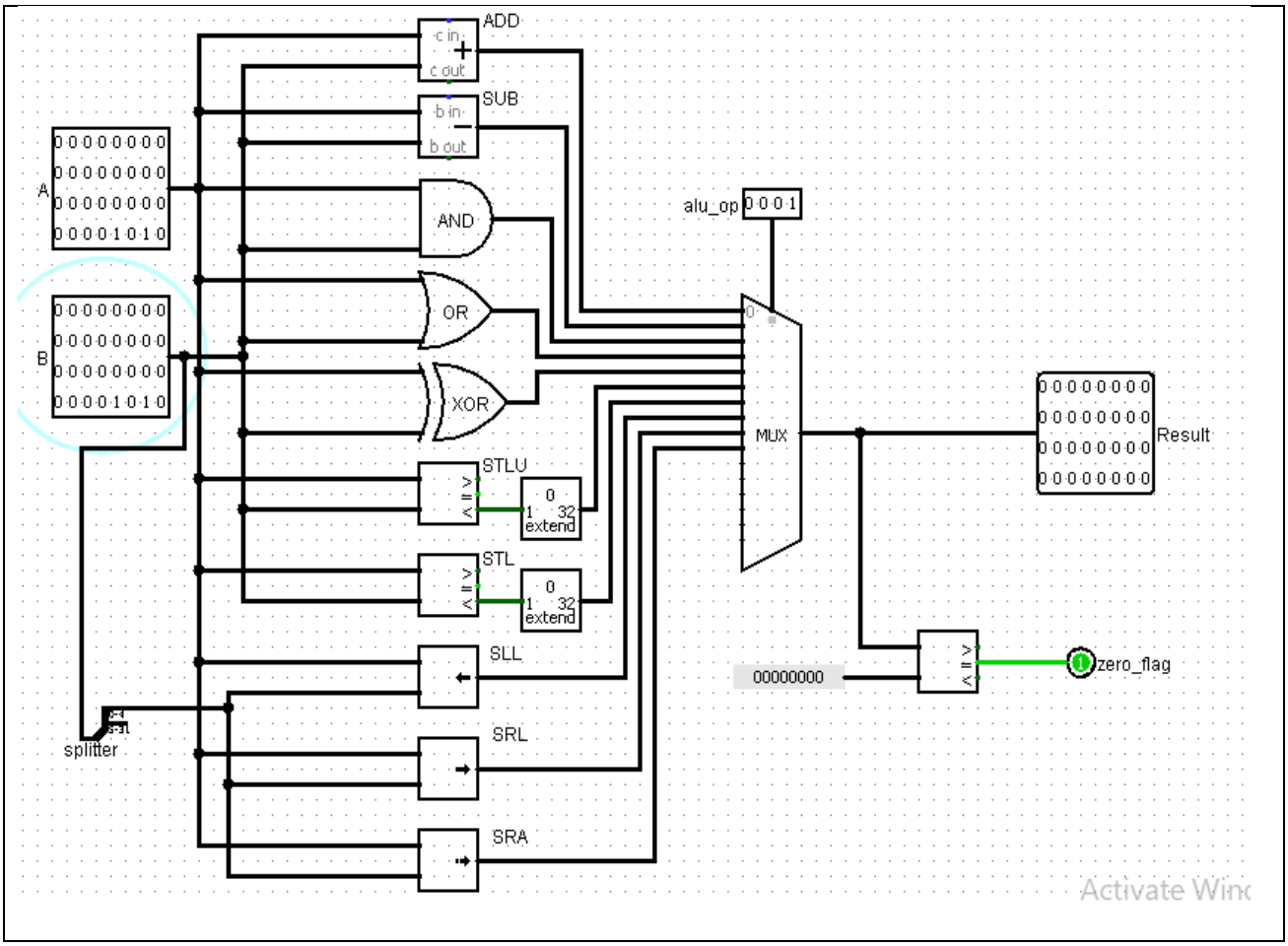
    x = -32'sd16; y = 32'd2; operation = 4'b1001; #10;

  end

endmodule
```

Output:




ADD
c in
c out
SUB
b in
b out
A
B
AND
OR
XOR
STLU
STL
SLL
SRL
SRA
extend
alu_op
MUX
Result
zero_flag
00000000
splitter